

DMN vs JDM — Research Findings

1. What They Are

DMN (Decision Model and Notation)

- OMG standard (same body as BPMN/UML), currently DMN 1.6 (April 2026)
- XML-based interchange format
- Uses FEEL (Friendly Enough Expression Language) for expressions
- Decision Requirement Diagrams (DRDs) show how decisions/inputs relate
- Backed by Camunda, Drools/KIE, IBM ODM, Trisotech, and others

JDM (JSON Decision Model)

- JSON-based format used primarily by GoRules' zen-engine (open source, Rust core)
- Lightweight, git-friendly, easy to embed
- Not an OMG/ISO standard — de facto format tied to the GoRules ecosystem
- Simple, JS-like expression syntax rather than FEEL

Practical Differences That Matter for Choosing

	DMN	JDM
Standardization	OMG standard, multi-vendor	Single-ecosystem (GoRules/zen-engine)
Format	XML	JSON
Expression language	FEEL	Simple expression syntax (JS-like)
Tooling maturity	Mature, enterprise-grade (Camunda, Drools, ODM)	Newer, lighter, growing fast
Embeddability	Heavier runtime	Very lightweight, easy to embed (Rust core, WASM-capable)
Best fit	Enterprise BPM stacks, regulated industries needing portability/audit trail	Developer-first apps wanting a fast, git-friendly, embeddable rules engine

2. Runtime Deployment Model

	Embedded	Separate Service
JDM (zen-engine)	Native default — official Go binding (gorules/zen-go , cgo over Rust) and Python binding (zen-engine via PyO3). Loads JSON rule graph in-process, no network hop.	Optional — GoRules also offers a hosted/self-hosted engine + visual editor for centralized management.
DMN	Possible via Drools/KIE embedded as a Java library, or Camunda's standalone DMN engine JAR — but this still requires a JVM.	Default in practice for enterprise engines (Camunda, IBM ODM) — centralized governance, business-user authoring, non-developers can publish rule changes.

Key constraint for a Go/Python stack: every mature DMN engine (Camunda, Drools/KIE, IBM ODM) is JVM-based. There is no production-grade native Go or Python DMN engine — DMN in a Go/Python shop almost always means calling a JVM-based decision service over REST/gRPC.

3. Enterprise Backing Comparison

DMN

- OMG-governed standard with a vendor-neutral conformance test suite (DMN TCK)
- Mid-2026 TCK conformance leaders: Goldman Sachs jDMN (100%), Trisotech (99.97%), IBM BAMOE and Apache KIE/Drools (99.91% each)
- Camunda's own conformance trails the pack (~84% DMN-Scala, ~81% Platform 7.21) — Camunda ≠ conformance leader
- Commercial ecosystem: IBM ODM, Red Hat Decision Manager, Pega, FICO Blaze Advisor, Progress Corticon, Camunda — full governance/audit/support tiers

JDM

- Single-vendor (GoRules), bootstrapped and customer-funded since 2023, no investors
- Used by Fortune 100 companies in financial services, insurance, logistics for compliance-sensitive rules
- Git-style branching, change requests, approval gates for audit trails; SOC 2 Type II

- No independent conformance suite, no second implementing vendor — GoRules explicitly wants tight control over the JDM spec/roadmap, not a multi-vendor standard

Bottom line: DMN wins on standards-body governance and multi-vendor conformance; JDM wins on operational fit for a Go/Python, JVM-free stack, backed by a single but credible vendor.

6. Overall Recommendation for the Go/Python Stack

If avoiding a JVM dependency is the priority: JDM/zen-engine is the pragmatic choice — native Go and Python bindings, JSON persistence, git-friendly, SOC 2, real Fortune 100 usage, no separate service required.

If vendor-neutral portability, FEEL, DRDs, or multi-vendor standards conformance are required (e.g., regulated industry, procurement checklist): DMN via a JVM-based decision service (Drools/KIE-backed) is the stronger fit, accepting the network hop and extra infra.